India Border Crossing Detection

BOLLU NIKHIL KUMAR
23311A05F3
III, CSE-C

ABSTRACT

Border monitoring is an important aspect of national security, and continuous manual surveillance is difficult and inefficient. This project presents a web-based system to detect and identify whether an individual crosses a predefined geographical border of India.

The system uses a geofencing approach with a ray-casting algorithm applied on a simplified India border polygon. GPS coordinates are tracked step by step, and the position is monitored across each step. When the tracked point crosses the predefined border zone, the system generates alerts indicating whether the person has entered or exited the border region.

The proposed system demonstrates the application of geographical computation and zone-based detection in border monitoring. It reduces manual effort, provides automated alerts through a real-time interactive UI built entirely with web technologies (HTML, CSS, JavaScript, and Leaflet.js), and can be extended to real-time GPS surveillance systems. The results show that the system effectively detects border crossing events and can serve as a foundation for advanced security applications.

LIST OF FIGURES			
S.NO	**Figure No.**	**Title of Figure**	**Page No**
1	6.1	Before entering into the Indian border	8
2	6.2	After entering into the Indian border	9
3	6.3	Before leaving into the Indian border	9
4	6.4	After leaving into the Indian border	9

Content	Page
Abstract	i
List of Figures	ii
INDEX	iii
CHAPTER - 1: INTRODUCTION	1
CHAPTER - 2: AIM OF THE PROJECT	2
CHAPTER - 3: DATA COLLECTION SOURCE AND INFORMATION ALONG WITH THE LINK USED	3
CHAPTER - 4: MODEL USED AND DESCRIPTION	4
CHAPTER - 5: SOURCE CODE	5 - 7
CHAPTER - 6: RESULTS	8 - 9
CHAPTER - 7: CONCLUSION	10
CHAPTER - 8: FUTURE SCOPE	11

INTRODUCTION

Border security is an important concern for any country, especially in sensitive areas where unauthorized entry or exit may occur. Manual monitoring of borders using human surveillance is difficult, time-consuming, and prone to errors. With the advancement of web technologies and geographical computation, automated systems can be developed to detect and monitor human movement in real time — accessible from any browser without installation.

This project presents a geofencing-based approach for detecting a person's GPS position and identifying whether the individual crosses a predefined geographical border of India. A simplified India border polygon is defined using key geographical vertices, and the movement of a tracked GPS point is monitored across consecutive steps. When the point crosses this boundary, the system generates alerts indicating entry into or exit from the border.

The proposed system provides a simple, efficient, and automated solution for border monitoring with a fully interactive visual UI powered by Leaflet.js. It works with predefined GPS routes covering four major border crossing scenarios. This approach reduces manual effort, improves monitoring accuracy, and demonstrates how geographical algorithms and modern web technologies can be applied in security and surveillance applications.

AIM OF THE PROJECT

The aim of this project is to design and develop a geofencing-based border monitoring system that detects a GPS-tracked position and identifies whether the individual crosses the India border using geographical boundary information. The system processes GPS coordinate input step by step, determines the zone of each coordinate using a ray-casting algorithm on India's border polygon, and tracks the movement of the detected position across a predefined virtual border.

The project also aims to generate real-time alerts when a person enters or exits the defined border region. By automating the surveillance process, the system reduces manual monitoring effort and improves accuracy in detecting unauthorized border crossing. Additionally, the project demonstrates the practical application of geographical computation and web-based visualization in security and surveillance systems.

The overall objective is to create an efficient, reliable, and scalable solution that can be further extended for real-time border monitoring using GPS devices, CCTV cameras, and other smart surveillance technologies — all accessible through a standard web browser.

DATA COLLECTION SOURCE AND INFORMATION ALONG WITH THE LINK USED

The data used for this project consists of GPS coordinate routes that simulate a person's movement across India's geographical border. The coordinate data is used as input for detecting movement and monitoring entry and exit across a virtual geographical boundary.

Routes are defined as sequences of latitude and longitude points. These routes contain movement from outside India to inside India and vice versa, which helps in testing both entry and exit detection scenarios.

Data Sources Used:

- Routes embedded in app.js — directly editable JavaScript object (originally from routes.csv)
- India border polygon coordinates sourced from publicly known geographical reference points
- Map tiles provided by CartoDB Dark Matter (via Leaflet.js CDN)
- Leaflet.js library: https://unpkg.com/leaflet@1.9.4/dist/leaflet.js
- Type of Data: GPS Coordinates (Latitude, Longitude)
- Format: JavaScript object (key → array of [lat, lon] pairs)
- Purpose: To simulate border crossing scenarios
- Preprocessing: Loaded directly at runtime in the browser — no server required

Available Routes:

Route Name	**Direction**
India → China (IND→CHN)	Delhi northward toward China border
Colombo → Chennai (LKA→IND)	Sri Lanka entering India from south
Delhi → Kathmandu (IND→NEP)	India exiting into Nepal
Kabul → Amritsar (AFG→IND)	Afghanistan entering India via Punjab

MODEL USED AND DESCRIPTION

The algorithm used in this project is the Ray-Casting Algorithm applied on a simplified India border polygon for zone detection, combined with a Geofencing Buffer Zone for early warning.

Ray-Casting Algorithm

The ray-casting algorithm determines whether a GPS point (latitude, longitude) lies inside or outside a polygon. It works by casting a horizontal ray from the point and counting how many times it intersects the polygon edges. An odd count means the point is inside; even means outside.

India Border Polygon (8 key vertices):

(37.1°N, 73.9°E) ← NW Kashmir

(35.5°N, 77.8°E) ← N Ladakh

(28.0°N, 97.3°E) ← NE Arunachal Pradesh

(22.0°N, 92.3°E) ← E Mizoram

(8.4°N, 77.5°E) ← S Kanyakumari

(8.5°N, 76.9°E) ← SW Kerala

(20.0°N, 68.7°E) ← W Gujarat

(24.0°N, 68.1°E) ← NW Rajasthan

Zone Classification:

Zone	Color	Condition
INSIDE	Green	Point is inside the India
BUFFER	Yellow	Within 2.0° (~222 km) of the border
OUTSIDE	Red	Beyond the buffer zone

SOURCE CODE

index.html — Main Page Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>🛡️ India Border Surveillance</title>
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
  <style>
    /* Dark theme, responsive two-column layout */
        body { background: #0d1117; color: #e6edf3; font-family: "Courier New",
monospace; }
    #map { width: 100%; height: 440px; border: 1px solid #58a6ff; }
    /* ... full styles in file ... */
  </style>
</head>
<body>
  <header>🛡️ INDIA BORDER SURVEILLANCE SYSTEM</header>
  <div class="main-row">
    <div class="map-col">
      <div id="map"></div>
      <div id="coordBar">Lat: --  Lon: --</div>
    </div>
    <div class="panel-col">
      <select id="routeSelect"></select>
      <input type="range" id="speedSlider" min="200" max="2000" value="900" />
      <button id="startBtn">▶ START TRACKING</button>
      <!-- Zone legend, status, event log -->
    </div>
```

```
  </div>
  <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

app.js — Core Logic

```
// ── Zone geometry ──────────────────────────────────────────────
const POLYGON = [
  [37.1, 73.9], [35.5, 77.8], [28.0, 97.3], [22.0, 92.3],
  [8.4,  77.5], [8.5,  76.9], [20.0, 68.7], [24.0, 68.1]
];
const BOUNDS  = { latMin: 8.4, latMax: 37.6, lonMin: 68.7, lonMax: 97.4 };
const BUFFER  = 2.0;

// Ray-casting point-in-polygon test
function getZone(lat, lon) {
  let inside = false;
  const n = POLYGON.length;
  for (let i = 0, j = n - 1; i < n; j = i++) {
    const [yi, xi] = POLYGON[i];
    const [yj, xj] = POLYGON[j];
    if ((yi > lat) !== (yj > lat) &&
        lon < ((xj - xi) * (lat - yi)) / (yj - yi) + xi) {
      inside = !inside;
    }
  }
  if (inside) return "INSIDE";
  return (lat >= latMin - BUFFER && lat <= latMax + BUFFER &&
          lon >= lonMin - BUFFER && lon <= lonMax + BUFFER)
    ? "BUFFER" : "OUTSIDE";
}
```

```
// Leaflet map with dark tile layer
const map = L.map("map", { center: [22, 82], zoom: 4 });
L.tileLayer("https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png", {
  attribution: '© CARTO', subdomains: "abcd"
}).addTo(map);

// India polygon and buffer zone overlays
L.polygon(POLYGON, { color: "#00ff88", fillColor: "#003300" }).addTo(map);
L.rectangle([[latMin-BUFFER, lonMin-BUFFER],[latMax+BUFFER,
lonMax+BUFFER]],
  { color: "#ffd600", dashArray: "6 4", fill: false }).addTo(map);

// Animated step-by-step tracking
function step(pts, i, prevZone) {
  if (i >= pts.length) { /* tracking complete */ return; }
  const [lat, lon] = pts[i];
  const zone  = getZone(lat, lon);
  const color = COLORS[zone];
  // Draw trail segment, move dot marker, log zone, fire alert on crossing
  setTimeout(() => step(pts, i + 1, zone), speedSlider.value);
}
```

RESULTS

The system successfully detects the GPS position and generates alerts when the position crosses the India border. The zone detection works accurately using the ray-casting algorithm.

The detection works in both directions: - Entry detection — GPS point moves from OUTSIDE → BUFFER → INSIDE India - Exit detection — GPS point moves from INSIDE → BUFFER → OUTSIDE India

Sample Output — Route: India → China (IND→CHN)

Step 1 28.61°N 77.20°E → ✅ Inside India

Step 2 30.00°N 79.00°E → ✅ Inside India

Step 3 31.50°N 81.00°E → ✅ Inside India

Step 4 33.00°N 83.00°E → ✅ Inside India

Step 5 34.50°N 85.00°E

🚨 BORDER CROSSED! Step 5

→ ⚠️ Buffer Zone — WARNING

── Tracking complete ──

Figure 6.1. Before entering into the Indian border

Figure 6.2. **After entering into the Indian border**

Figure 6.3. **Before leaving into the Indian border**

Figure 6.4. **After leaving into the Indian border**

CONCLUSION

This project successfully implements a geofencing-based border monitoring system that detects a GPS-tracked position and identifies whether the individual crosses a predefined geographical boundary of India. By using the ray-casting algorithm on a simplified India border polygon, the system accurately determines the zone of each GPS coordinate and tracks movement across a virtual border line.

The developed system generates alerts when a person enters or exits the border, demonstrating real-time surveillance capability through an animated, dark-themed interactive web UI built with HTML, CSS, JavaScript, and Leaflet.js. The implementation reduces manual monitoring effort and improves detection efficiency.

The results show that the system performs effectively for all defined routes, correctly identifying INSIDE, BUFFER, and OUTSIDE zones. The buffer zone provides an early warning layer before the actual border is crossed.

The web-based conversion brings additional advantages over the original Python/ Tkinter desktop application — it runs in any browser without installation, features a real interactive map with zoom and pan, and includes a speed control slider for flexible playback. Overall, the project demonstrates how geographical computation and modern web visualization can be applied to security and border monitoring applications. The system is simple, efficient, and can be extended for real-world deployment with live GPS feeds and advanced tracking features.

FUTURE SCOPE

The proposed border crossing detection system can be further enhanced and extended in several ways to improve its performance and real-world applicability.

- Real-time GPS integration — Connect to live GPS devices or mobile phone location APIs (e.g., browser Geolocation API) for continuous real-time border monitoring instead of simulated routes.
- Precise border polygon — Replace the simplified 8-point polygon with a detailed GeoJSON shapefile from Natural Earth or Survey of India for exact border accuracy, loaded directly into Leaflet.
- Multiple person tracking — Extend the system to monitor several individuals simultaneously, each with their own GPS track, color-coded trail, and alert.
- Face recognition integration — Incorporate face recognition techniques via a backend API to identify individuals crossing the border and match against a database.
- CCTV camera integration — Combine with YOLOv8 object detection on live camera feeds to detect and track persons visually, feeding coordinates into the web map.
- Alert system — Connect border crossing alerts to browser push notifications, SMS gateways, or security dashboards for immediate response.
- Drone-based surveillance — Extend the system to process GPS telemetry from drones monitoring large border areas, visualized in real time on the Leaflet map.
- Advanced tracking algorithms — Use Kalman filters or deep learning-based trackers to improve accuracy in noisy GPS environments.
- CSV / GPX file upload — Allow users to upload their own route files directly in the browser, replacing the hardcoded route data.